

ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN – TIN



Báo cáo Bài tập lớn cuối kỳ

Cấu trúc dữ liệu và Giải thuật

Hệ thống quản lý bệnh viện sử dụng hàng đợi ưu tiên

GVHD: Vương Mai Phương

Nhóm 9

Đặng Tuấn Đạt – 202418867

Nguyễn Mai Đức – 202418870

Nguyễn Huy Đức – 202418871

Nguyễn Bá Dũng – 202418877

Nguyễn Hải Dương – 20227102

Hà Nội, tháng 6 năm 2026

Mục lục

Lời mở đầu	4
1 Phân tích bài toán	5
1.1 Bối cảnh thực tế	5
1.2 Yêu cầu chức năng	5
1.3 Yêu cầu phi chức năng	7
1.4 Phát biểu bài toán	7
1.5 Lý do không dùng danh sách thông thường	8
2 Thiết kế hệ thống và cài đặt chương trình	9
2.1 Kiến trúc tổng thể và phân chia module	9
2.2 Sơ đồ lớp (Class Diagram)	10
2.3 Cấu trúc dữ liệu	11
2.3.1 Lớp Patient – kiểu dữ liệu và quy tắc so sánh ưu tiên	11
2.3.2 ADT MinHeap – hàng đợi ưu tiên lỗi	12
2.3.3 ADT SinglyLinkedList – lưu lịch sử khám	14
2.3.4 ADT PatientHashMap – registry tra cứu nhanh	15
2.4 Giải mã các thuật toán	17
2.4.1 loadData – Đọc dữ liệu từ file	17
2.4.2 saveData – Ghi dữ liệu xuống file	17
2.4.3 Thêm bệnh nhân – enqueue / heapify_up	18
2.4.4 Gọi bệnh nhân vào khám – dequeue / heapify_down	18
2.4.5 Tìm kiếm và xóa bệnh nhân	19
2.5 Phân tích độ phức tạp	20
2.5.1 Độ phức tạp thời gian	20
2.5.2 Độ phức tạp không gian	20
3 Giao diện người dùng	21
3.1 Tổng quan thiết kế giao diện	21
3.2 Màn hình menu chính	22
3.3 Mô tả các chức năng	23

3.4	Xử lý lỗi và kiểm tra đầu vào	24
4	Thử nghiệm và đánh giá hiệu năng	25
4.1	Kịch bản thực nghiệm	25
4.2	Kết quả thực nghiệm	25
4.3	Phân tích và nhận xét	25
5	Kiểm thử chương trình	27
5.1	Kịch bản kiểm thử	27
5.2	Kết quả kiểm thử	29
6	Kết luận và hướng phát triển	30
6.1	Kết luận	30
6.2	Hạn chế và hướng phát triển	30
7	Phân công và đánh giá nhóm	31
	Tài liệu tham khảo	32

Danh sách bảng

1.1	So sánh các cấu trúc dữ liệu cho bài toán hàng đợi ưu tiên	8
2.1	Cấu trúc thư mục và vai trò module	9
2.2	Thuộc tính lớp <code>Patient</code>	12
2.3	Các phương thức <code>MinHeap</code> và độ phức tạp	14
2.4	Các phương thức <code>SinglyLinkedList</code> và độ phức tạp	15
2.5	Các phương thức <code>PatientHashMap</code> và độ phức tạp	16
2.6	Bảng tổng hợp độ phức tạp thời gian	20
2.7	Bảng tổng hợp độ phức tạp không gian	20
3.1	Tóm tắt các chức năng giao diện và phương thức <code>Scheduler</code> tương ứng . . .	22
3.2	Các hàm validate đầu vào tại tầng UI	24
4.1	Thời gian thực thi (N enqueue + N dequeue, đơn vị: ms)	25
5.1	Bảng kịch bản kiểm thử hệ thống (26 test case)	27
7.1	Bảng phân công chi tiết công việc thành viên	31

Lời mở đầu

Đề tài *Hệ thống quản lý bệnh viện sử dụng hàng đợi ưu tiên* được lựa chọn dựa trên mục đích xây dựng hệ thống quản lý và phân loại bệnh nhân bằng cấu trúc hàng đợi ưu tiên, vốn được thực hiện thủ công dẫn đến dễ sai sót và khó bảo trì

Mục tiêu của bài toán là xây dựng chương trình có khả năng thêm bệnh nhân, gọi bệnh nhân ưu tiên nhất vào khám, tìm kiếm, sửa, xóa thông tin và lưu lịch sử khám. Nhóm tiếp cận bài toán bằng cách phân tích chức năng, thiết kế cấu trúc dữ liệu, sau đó triển khai bằng ngôn ngữ lập trình Python.

Báo cáo được tổ chức thành các chương: Chương 1 phân tích bài toán; Chương 2 trình bày thiết kế hệ thống và cài đặt chương trình; Chương 3 giới thiệu giao diện người dùng; Chương 4 trình bày kết quả thực nghiệm; Chương 5 trình bày kiểm thử; Chương 6 kết luận và hướng phát triển.

Chương 1

Phân tích bài toán

Bài toán đặt ra là thiết kế và triển khai một hệ thống quản lý hàng đợi bệnh nhân của một bệnh viện. Hệ thống cần giải quyết được các vấn đề như số lượng bệnh nhân khổng lồ, đảm bảo ưu tiên bệnh nhân nguy kịch đồng thời duy trì tính công bằng giữa các bệnh nhân có cùng mức độ ưu tiên

1.1 Bối cảnh thực tế

Tại các phòng cấp cứu và phòng khám bệnh viện, số lượng bệnh nhân đến khám mỗi ngày thường rất lớn và không đến theo thứ tự mức độ nguy hiểm. Thực tế cho thấy một bệnh nhân bị đau tim có thể đến sau một bệnh nhân chỉ bị cảm nhẹ – nếu áp dụng FIFO (đến trước khám trước) thì bệnh nhân nguy kịch vẫn phải chờ, gây nguy hiểm đến tính mạng.

Hệ thống cần xây dựng là một phần mềm có khả năng:

- Tự động xếp thứ tự bệnh nhân theo mức độ ưu tiên y tế.
- Đảm bảo người nguy kịch luôn được khám trước.
- Duy trì tính công bằng FIFO khi hai bệnh nhân có cùng mức độ bệnh (ai đến trước thì được khám trước).

Đây là bài toán điển hình của Priority Queue (hàng đợi ưu tiên) trong khoa học máy tính, ứng dụng thực tiễn vào lĩnh vực y tế.

1.2 Yêu cầu chức năng

1. Quản lý hàng đợi bệnh nhân:

- Lưu trữ thông tin mỗi bệnh nhân gồm: mã số (tự động sinh tăng dần), họ tên, tuổi, mức độ bệnh (1–5), giờ đến.

- Cho phép cập nhật thông tin khi có sai sót (ví dụ: nhập nhầm mức độ bệnh ban đầu).
- Cho phép xóa bệnh nhân khỏi hàng đợi (ví dụ: bệnh nhân tự về hoặc chuyển viện trước khi được khám).
- Hỗ trợ tìm kiếm theo mã số hoặc theo tên bệnh nhân.

2. Gọi bệnh nhân vào khám: Lấy bệnh nhân ưu tiên nhất khỏi hàng đợi. Quy tắc ưu tiên 3 cấp:

- Cấp 1: Mức độ bệnh nhỏ hơn $\Rightarrow$ ưu tiên hơn (1 = nguy kịch, 2 = nặng, 3 = trung bình, 4 = nhẹ, 5 = rất nhẹ).
- Cấp 2: Nếu cùng mức độ $\Rightarrow$ người đến sớm hơn $\Rightarrow$ ưu tiên hơn (đảm bảo tính công bằng FIFO).
- Cấp 3: Nếu cùng cả mức độ lẫn giờ đến $\Rightarrow$ số đăng ký nhỏ hơn $\Rightarrow$ ưu tiên hơn (tie-breaker).

Bệnh nhân sau khi được gọi sẽ bị xóa khỏi hàng đợi và chuyển sang danh sách lịch sử.

3. Quản lý lịch sử khám:

- Ghi lại toàn bộ bệnh nhân đã khám theo đúng thứ tự thời gian thực tế được gọi vào phòng khám.
- Lịch sử giúp tra cứu lại ca khám, phục vụ kiểm tra hoặc báo cáo cuối ngày.

4. Thống kê:

- Báo cáo tổng số bệnh nhân đang chờ và đã khám.
- Phân loại theo từng mức độ bệnh (1–5) để ban quản lý nắm được tình hình tổng thể của phòng khám.

5. Lưu trữ dữ liệu (`loadData` / `saveData`):

- Sau mỗi thao tác thay đổi dữ liệu, hệ thống tự động ghi xuống file CSV.
- Khi khởi động lại chương trình, dữ liệu được đọc từ file và khôi phục nguyên vẹn hàng đợi và lịch sử.
- Mục đích: đảm bảo không mất dữ liệu khi tắt chương trình đột ngột (mất điện, lỗi hệ thống...).

1.3 Yêu cầu phi chức năng

Ngoài các chức năng trên, hệ thống còn phải đáp ứng các tiêu chí kỹ thuật sau:

- **Hiệu suất:** Các thao tác cốt lõi (thêm bệnh nhân, gọi khám) phải có độ phức tạp thời gian $O(\log N)$, đảm bảo phản hồi nhanh ngay cả khi hàng đợi chứa hàng nghìn bệnh nhân. Với $N = 10.000$, mỗi thao tác chỉ mất khoảng 13 bước tính toán thay vì 10.000 bước như danh sách thông thường.
- **Độ tin cậy:** Dữ liệu không được mất sau khi tắt chương trình. Mọi thay đổi phải được ghi xuống file ngay lập tức, không chờ đến khi thoát.
- **Khả năng mở rộng:** Kiến trúc được tổ chức thành các lớp riêng biệt, tách biệt dữ liệu, logic xử lý và giao diện người dùng. Nhờ đó có thể dễ dàng bổ sung tính năng mới mà không phải viết lại toàn bộ chương trình.

1.4 Phát biểu bài toán

Đầu vào:

- Danh sách bệnh nhân đến khám, mỗi người gồm: họ tên, tuổi, mức độ bệnh (1–5), giờ đến (định dạng HH:MM).
- Các thao tác người dùng thực hiện: thêm bệnh nhân, gọi khám, sửa thông tin, xóa, tìm kiếm.

Đầu ra:

- Mỗi lần gọi khám, hệ thống trả về đúng bệnh nhân có độ ưu tiên cao nhất theo quy tắc 3 cấp.
- Lịch sử khám được lưu đầy đủ và có thể truy xuất lại.
- Toàn bộ dữ liệu được lưu xuống file sau mỗi thay đổi, không mất khi tắt chương trình.

Ràng buộc:

- Mức độ bệnh là số nguyên từ 1 đến 5, số càng nhỏ thì mức độ ưu tiên càng cao (1 = nguy kịch, 5 = rất nhẹ).
- Khi hai bệnh nhân có cùng mức độ bệnh, người đến sớm hơn được ưu tiên (FIFO).
- Khi cùng cả mức độ lẫn giờ đến, dùng số thứ tự đăng ký nhỏ hơn làm tie-breaker để đảm bảo hệ thống luôn có kết quả xác định.

1.5 Lý do không dùng danh sách thông thường

Khi xây dựng hàng đợi ưu tiên, hai cách tiếp cận đơn giản nhất là dùng danh sách chưa sắp xếp hoặc danh sách đã sắp xếp. Tuy nhiên cả hai đều có điểm yếu rõ ràng:

- **Unsorted List:** Thêm mới $O(1)$ nhưng mỗi lần gọi khám phải duyệt toàn bộ để tìm người ưu tiên nhất – $O(N)$. Với 10.000 bệnh nhân, mỗi lần bác sĩ nhấn “gọi khám” hệ thống phải duyệt 10.000 bản ghi.
- **Sorted List:** Gọi khám $O(1)$ nhưng thêm mới phải tìm vị trí đúng và dịch mảng – $O(N)$. Mỗi lần có bệnh nhân mới đăng ký, hệ thống phải sắp xếp lại toàn bộ.

Binary Min-Heap giải quyết cả hai: thêm mới $O(\log N)$, lấy phần tử ưu tiên nhất $O(\log N)$. Heap được lưu dưới dạng mảng, với nút tại vị trí i :

$$\text{cha}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor, \quad \text{con trái} = 2i+1, \quad \text{con phải} = 2i+2$$

Bảng 1.1: So sánh các cấu trúc dữ liệu cho bài toán hàng đợi ưu tiên

Tiêu chí	Unsorted List	Sorted List	Binary Min-Heap
Thêm bệnh nhân	$O(1)$	$O(N)$	$O(\log N)$
Gọi khám	$O(N)$	$O(1)$	$O(\log N)$
$N = 10.000$ (worst)	10.000 bước	10.000 bước	~ 13 bước ✓
Bộ nhớ	$O(N)$	$O(N)$	$O(N)$
Phù hợp bài toán	Không	Không	Có ✓

Chương 2

Thiết kế hệ thống và cài đặt chương trình

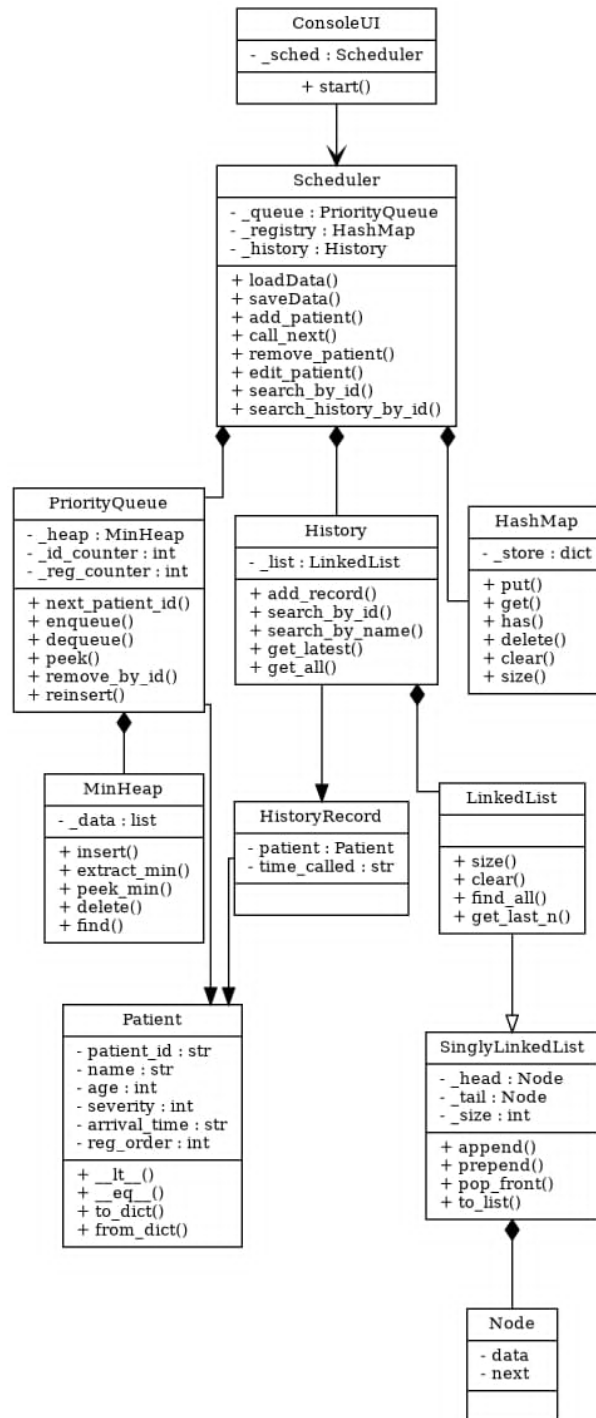
2.1 Kiến trúc tổng thể và phân chia module

Hệ thống được tổ chức thành bốn tầng tách biệt. Mỗi tầng chỉ giao tiếp với tầng ngay dưới, không truy cập trực tiếp vào thuộc tính nội bộ của tầng khác.

Bảng 2.1: Cấu trúc thư mục và vai trò module

Tầng	Module / File	Vai trò
Data	Data/patient.py	Kiểu dữ liệu <code>Patient</code> , quy tắc so sánh ưu tiên
DataStructures	DataStructures/min_heap.py	ADT Min-Heap tự cài đặt
	DataStructures/linked_list.py	ADT Linked List đơn
	DataStructures/hash_map.py	ADT HashMap (bảng băm)
Algorithm	Algorithm/priority_queue.py	Hàng đợi ưu tiên bệnh nhân dùng MinHeap
	Algorithm/history_tracker.py	Lịch sử khám dùng LinkedList
	Algorithm/scheduler.py	Điều phối toàn bộ nghiệp vụ
UI	UI/main_console.py	Giao diện console người dùng

2.2 Sơ đồ lớp (Class Diagram)



Hình 2.1: Sơ đồ lớp hệ thống quản lý bệnh viện

2.3 Cấu trúc dữ liệu

Mục này trình bày cài đặt bốn thành phần cốt lõi theo thứ tự phân tầng từ dưới lên: `Patient` (tầng Data), `MinHeap`, `SinglyLinkedList` và `PatientHashMap` (tầng DataStructures).

2.3.1 Lớp Patient – kiểu dữ liệu và quy tắc so sánh ưu tiên

`MinHeap` không biết tới "patient" – nó chỉ gọi toán tử so sánh; `Patient` tự định nghĩa toán tử đó để quyết định thứ tự ưu tiên.

Listing 2.1: Khởi tạo và quy tắc so sánh ưu tiên 3 cấp (patient.py)

```
1 class Patient:
2     def __init__(self, patient_id, name, age, severity,
3         arrival_time, reg_order=0):
4         self.patient_id = str(patient_id).strip()
5         self.name = str(name).strip()
6         self.age = int(age)
7         self.severity = int(severity)
8         self.arrival_time = str(arrival_time).strip()
9         self.reg_order = int(reg_order)
10
11     def __lt__(self, other):
12         if self.severity != other.severity:
13             return self.severity < other.severity      # cap 1
14         if self.arrival_time != other.arrival_time:
15             return self.arrival_time < other.arrival_time # cap 2
16         return self.reg_order < other.reg_order         # cap 3
```

Ba cấp ưu tiên: (1) **severity** – số càng nhỏ càng nặng; (2) **arrival_time** – cùng mức độ thì đến trước khám trước (FIFO); (3) **reg_order** – số thứ tự đăng ký, không reset khi khởi động lại, đảm bảo luôn so sánh được.

arrival_time được so sánh như chuỗi khi format đồng nhất HH:MM. Hàm validate_arrival_time() tự động chuẩn hoá "9:00" thành "09:00":

Listing 2.2: Tự động chuẩn hoá định dạng giờ (patient.py)

```

1 _TIME_RE = re.compile(r'^([01]\d|2[0-3]):[0-5]\d$')
2
3 def validate_arrival_time(time_str: str) -> tuple[bool, str]:
4     s = str(time_str).strip()
5     if _TIME_RE.match(s):
6         return True, s
7     if re.match(r'^\d:[0-5]\d$', s):
8         fixed = '0' + s
9         if _TIME_RE.match(fixed):
10            return True, fixed
11    return False, f"Gio den '{time_str}' không hợp lệ."

```

Bảng 2.2: Thuộc tính lớp Patient

Thuộc tính	Kiểu	Mô tả
patient_id	str	Mã tự động, ví dụ BN0001
name	str	Họ và tên
age	int	Tuổi
severity	int	Mức độ bệnh 1 (nguy kịch) – 5 (rất nhẹ)
arrival_time	str	Giờ đến, định dạng HH:MM
reg_order	int	Số thứ tự đăng ký, dùng làm tie-breaker

2.3.2 ADT MinHeap – hàng đợi ưu tiên lỗi

MinHeap là cấu trúc thuần túy – toàn bộ dữ liệu nằm trong mảng `_data`, quan hệ cha-con tính bằng chỉ số, không dùng con trỏ.

Listing 2.3: insert và `__heapify_up`: $O(\log N)$ (min_heap.py)

```

1 def insert(self, item):
2     self._data.append(item)
3     self._heapify_up(len(self._data) - 1)
4
5 def __heapify_up(self, i):
6     parent = (i - 1) // 2
7     if i > 0 and self._data[i] < self._data[parent]:
8         self._data[i], self._data[parent] = \
9             self._data[parent], self._data[i]
10    self._heapify_up(parent)

```

Mỗi bước chỉ số i được chia đôi để lên cha; tối đa $\lfloor \log_2 N \rfloor$ bước $\Rightarrow O(\log N)$.

Listing 2.4: extract_min và _heapify_down: $O(\log N)$ (min_heap.py)

```

1 def extract_min(self):
2     if self.is_empty(): return None
3     if len(self._data) == 1: return self._data.pop()
4     minimum = self._data[0]
5     self._data[0] = self._data.pop() # tránh pop() O(N)
6     self._heapify_down(0)
7     return minimum
8
9 def _heapify_down(self, i):
10    smallest, n = i, len(self._data)
11    left, right = 2*i + 1, 2*i + 2
12    if left < n and self._data[left] < self._data[smallest]:
13        smallest = left
14    if right < n and self._data[right] < self._data[smallest]:
15        smallest = right
16    if smallest != i:
17        self._data[i], self._data[smallest] = \
18            self._data[smallest], self._data[i]
19        self._heapify_down(smallest)

```

Thao tác delete theo predicate tìm vị trí $O(N)$, sau đó đổi chỗ với phần tử cuối và gọi cả _heapify_up lẫn _heapify_down để phục hồi heap:

Listing 2.5: delete theo predicate (min_heap.py)

```

1 def delete(self, predicate):
2     idx = next((i for i, x in enumerate(self._data)
3                 if predicate(x)), -1)
4     if idx == -1: return None
5     removed = self._data[idx]
6     if idx == len(self._data) - 1:
7         self._data.pop()
8     else:
9         self._data[idx] = self._data.pop()
10        self._heapify_up(idx)
11        self._heapify_down(idx)
12    return removed

```

Bảng 2.3: Các phương thức MinHeap và độ phức tạp

Phương thức	Mô tả	Big O
<code>insert(item)</code>	Thêm vào cuối + <code>_heapify_up</code>	$O(\log N)$
<code>extract_min()</code>	Lấy đỉnh, đưa cuối lên + <code>_heapify_down</code>	$O(\log N)$
<code>peek_min()</code>	Trả về <code>data[0]</code> , không xóa	$O(1)$
<code>delete(predicate)</code>	Tìm tuyến tính + xóa + sửa heap	$O(N)$
<code>find(predicate)</code>	Duyệt toàn bộ mảng	$O(N)$
<code>to_sorted_list()</code>	Sao chép + sắp xếp	$O(N \log N)$
<code>is_empty()</code> , <code>size()</code>	Truy cập trực tiếp	$O(1)$
<code>_heapify_up(i)</code>	Đẩy lên tối đa $\log N$ tầng	$O(\log N)$
<code>_heapify_down(i)</code>	Đẩy xuống tối đa $\log N$ tầng	$O(\log N)$

2.3.3 ADT SinglyLinkedList – lưu lịch sử khám

Lịch sử khám chỉ cần *append* $O(1)$ và *duyệt tuần tự* $O(N)$. Con trỏ `_tail` riêng biệt là điểm mấu chốt – *append* không phải duyệt đến cuối:

Listing 2.6: Node và *append* $O(1)$ nhờ con trỏ `_tail` (`linked_list.py`)

```

1 class Node:
2     __slots__ = ('data', 'next')
3     def __init__(self, data):
4         self.data = data; self.next = None
5
6 class SinglyLinkedList:
7     def __init__(self):
8         self._head = self._tail = None
9         self._size = 0
10
11     def append(self, data) -> None:
12         new_node = Node(data)
13         if self._tail is None:
14             self._head = self._tail = new_node
15         else:
16             self._tail.next = new_node
17             self._tail = new_node
18         self._size += 1

```

Bảng 2.4: Các phương thức SinglyLinkedList và độ phức tạp

Phương thức	Mô tả	Big O
append(data)	Thêm vào cuối, nhờ con trỏ tail	$O(1)$
prepend(data)	Thêm vào đầu, cập nhật head	$O(1)$
find_first(p)	Duyệt từ head, trả về kết quả đầu tiên	$O(N)$
find_all(p)	Duyệt toàn bộ, trả về list kết quả	$O(N)$
to_list()	Duyệt từ head đến tail	$O(N)$
size(),	Truy cập biến _size	$O(1)$
is_empty()		
clear()	Đặt head, tail = None; _size = 0	$O(1)$

2.3.4 ADT PatientHashMap – registry tra cứu nhanh

MinHeap không hỗ trợ random access theo ID ($O(N)$ để tìm). PatientHashMap giải quyết bằng cơ chế băm: `hash("BN0042")` → chỉ số slot → $O(1)$.

Lớp duy trì song song: `_store` (dict CPython) cho production path, và `_demo` (`_ChainHashMap` tự cài đặt Separate Chaining) để minh họa cơ chế:

Listing 2.7: Separate Chaining và rehash trong `_ChainHashMap` (`hash_map.py`)

```

1 class _ChainHashMap:
2     _LOAD_MAX = 0.75
3
4     def _index(self, key: str) -> int:
5         return abs(hash(key)) % self._cap
6
7     def put(self, key: str, value) -> None:
8         idx = self._index(key)
9         bucket = self._table[idx]
10        for i, (k, _) in enumerate(bucket):
11            if k == key:
12                bucket[i] = (key, value); return
13        bucket.append((key, value))
14        self._size += 1
15        if self._size / self._cap > self._LOAD_MAX:
16            self._rehash()
17
18        def _rehash(self) -> None:           # O(N), hiếm gap
19            old_table = self._table
20            self._cap *= 2
21            self._table = [[] for _ in range(self._cap)]
22            self._size = 0

```



```

23     for bucket in old_table:
24         for key, value in bucket:
25             self.put(key, value)

```

Khi load factor $\alpha > 0,75$, `_rehash` nhân đôi mảng và băm lại toàn bộ – $O(N)$ mỗi lần nhưng ngày càng ít xảy ra $\Rightarrow$ *amortised* $O(1)$ cho mỗi `put`.

Listing 2.8: register, lookup, unregister trong PatientHashMap (hash_map.py)

```

1 class PatientHashMap:
2     def __init__(self):
3         self._store: dict = {}
4         self._demo = _ChainHashMap(16)
5
6     def register(self, patient) -> None:
7         if not hasattr(patient, 'patient_id'):
8             raise TypeError(
9                 f"Chi nhan Patient; nhan {type(patient).__name__}.")
10        self._store[patient.patient_id] = patient
11        self._demo.put(patient.patient_id, patient)
12
13    def lookup(self, patient_id: str):
14        return self._store.get(patient_id)
15
16    def unregister(self, patient_id: str) -> bool:
17        if patient_id in self._store:
18            del self._store[patient_id]
19            self._demo.delete(patient_id)
20            return True
21        return False

```

Phối hợp MinHeap và PatientHashMap: Khi thêm bệnh nhân, cả hai đều cập nhật đồng thời. Khi sửa/xóa, Scheduler tra map trước $O(1)$ rồi mới gọi `heap.delete()` $O(N)$. Nhờ vậy, không thao tác nào trên đường chính (thêm, gọi khám) vượt quá $O(\log N)$.

Bảng 2.5: Các phương thức PatientHashMap và độ phức tạp

Phương thức	Mô tả	Big O
<code>register(patient)</code>	Lưu hoặc ghi đè cặp (key, value)	$O(1)^*$
<code>lookup(key)</code>	Tra cứu theo key	$O(1)^*$
<code>unregister(key)</code>	Xóa cặp (key, value)	$O(1)^*$
<code>size()</code> , <code>is_empty()</code>	Số cặp hiện có	$O(1)$

* Trung bình. Trường hợp xấu nhất $O(N)$ khi nhiều va chạm hash.

2.4 Giải mã các thuật toán

2.4.1 loadData – Đọc dữ liệu từ file

Thuật toán 1 loadData()

Đầu vào: File `queue.csv` và `history.csv`

Đầu ra: Hàng đợi và lịch sử được nạp vào bộ nhớ

- 1: Mở file `queue.csv`
 - 2: **if** file không tồn tại **then**
 - 3: Kết thúc (hàng đợi bắt đầu rỗng)
 - 4: **end if**
 - 5: **for** mỗi dòng trong file **do**
 - 6: Phân tích dòng CSV thành các trường dữ liệu
 - 7: Tạo đối tượng `Patient` từ các trường đó
 - 8: `MINHEAP.INSERT(Patient)`
 - 9: **end for**
 - 10: Xây dựng lại registry: duyệt heap, `HASHMAP.PUT(id, Patient)`
 - 11: Đọc `history.csv` tương tự, `LINKEDLIST.APPEND(HistoryRecord)`
-

2.4.2 saveData – Ghi dữ liệu xuống file

Thuật toán 2 saveData()

Đầu vào: Hàng đợi và lịch sử hiện có trong bộ nhớ

Đầu ra: Dữ liệu được ghi ra `queue.csv` và `history.csv`

- 1: Mở file `queue.csv` để ghi (ghi đè)
 - 2: Ghi dòng tiêu đề CSV
 - 3: **for** mỗi `Patient` trong mảng nội bộ của `MinHeap` **do**
 - 4: Ghi dòng: id, tên, tuổi, mức độ, giờ đến, số đăng ký
 - 5: **end for**
 - 6: Mở file `history.csv` để ghi (ghi đè), ghi toàn bộ lịch sử
-

2.4.3 Thêm bệnh nhân – enqueue / heapify_up

Thuật toán 3 addPatient(name, age, severity, arrival_time)

Đầu vào: Thông tin bệnh nhân mới

Đầu ra: Bệnh nhân được thêm vào hàng đợi, dữ liệu được lưu

```
1: if thông tin không hợp lệ then
2:     Thông báo lỗi, kết thúc
3: end if
4:  $id \leftarrow$  sinh ID tự động dạng BNxxxx
5:  $reg \leftarrow$  tăng bộ đếm thứ tự đăng ký
6: Tạo đối tượng Patient(id, name, age, severity, arrival_time, reg)
7: MINHEAP.INSERT(Patient)
8: HASHMAP.PUT(id, Patient)
9: SAVEDATA
```

Thuật toán 4 heapify_up(i)

Đầu vào: i : vị trí phần tử vừa thêm (cuối mảng)

```
1:  $parent \leftarrow \lfloor (i - 1) / 2 \rfloor$ 
2: if  $i > 0$  và  $data[i] < data[parent]$  then
3:     Đổi chỗ  $data[i]$  và  $data[parent]$ 
4:     HEAPIFY_UP( $parent$ )
5: end if
```

Độ phức tạp: $O(\log N)$ – cây có chiều cao $\lfloor \log_2 N \rfloor$, heapify_up đi lên tối đa bấy nhiêu tầng.

2.4.4 Gọi bệnh nhân vào khám – dequeue / heapify_down

Thuật toán 5 callNext()

Đầu ra: Trả về bệnh nhân ưu tiên nhất; cập nhật lịch sử và file

```
1: if hàng đợi rỗng then
2:     Thông báo “không có bệnh nhân”, kết thúc
3: end if
4:  $p \leftarrow$  MINHEAP.EXTRACT_MIN
5: HASHMAP.DELETE( $p.id$ )
6: LINKEDLIST.APPEND(HistoryRecord( $p$ , giờ hiện tại))
7: SAVEDATA
8: return  $p$ 
```

Thuật toán 6 heapify_down(*i*)

Đầu vào: *i*: vị trí cần đẩy xuống; *N*: số phần tử

```
1: smallest ← i;   left ← 2i + 1;   right ← 2i + 2
2: if left < N và data[left] < data[smallest] then
3:   smallest ← left
4: end if
5: if right < N và data[right] < data[smallest] then
6:   smallest ← right
7: end if
8: if smallest ≠ i then
9:   Đổi chỗ data[i] và data[smallest]
10:  HEAPIFY_DOWN(smallest)
11: end if
```

Độ phức tạp: $O(\log N)$ – tương tự heapify_up, đi xuống tối đa $\log N$ tầng.

2.4.5 Tìm kiếm và xóa bệnh nhân

Thuật toán 7 searchById(*id*)

```
1: return HASHMAP.GET(id)
```

▷ $O(1)$ trung bình

Thuật toán 8 removePatient(*id*)

```
1: if không HASHMAP.HAS(id) then
2:   Thông báo “không tìm thấy”, kết thúc
3: end if
4: Duyệt mảng heap, tìm vị trí i có data[i].id = id
5: Đổi data[i] với phần tử cuối, xóa phần tử cuối
6: HEAPIFY_UP(i);   HEAPIFY_DOWN(i)
7: HASHMAP.DELETE(id)
8: SAVEDATA
```

2.5 Phân tích độ phức tạp

2.5.1 Độ phức tạp thời gian

Bảng 2.6: Bảng tổng hợp độ phức tạp thời gian

Thao tác	Cấu trúc	Thời gian	Giải thích
Thêm bệnh nhân	MinHeap	$O(\log N)$	Heapify up theo chiều cao cây
Gọi khám	MinHeap	$O(\log N)$	Heapify down theo chiều cao cây
Xem tiếp theo	MinHeap	$O(1)$	Luôn ở <code>data[0]</code>
Tìm theo ID	HashMap	$O(1)$	Tra cứu trực tiếp qua mã băm
Xóa/Sửa thông tin	MinHeap	$O(N)$	Duyệt mảng tìm vị trí phần tử
Lưu lịch sử	LinkedList	$O(1)$	Sử dụng con trỏ <i>tail</i>
loadData	–	$O(N \log N)$	Đọc N dòng, mỗi dòng enqueue $O(\log N)$
saveData	–	$O(N)$	Duyệt toàn bộ, ghi từng dòng

2.5.2 Độ phức tạp không gian

Bảng 2.7: Bảng tổng hợp độ phức tạp không gian

Cấu trúc	Không gian	Giải thích
MinHeap (hàng đợi)	$O(N)$	Mảng N phần tử
HashMap (registry)	$O(N)$	N cặp (key, Patient)
LinkedList (lịch sử)	$O(M)$	M = số lượt đã khám
heapify_up/down (stack)	$O(\log N)$	Đệ quy sâu tối đa $\log N$ tầng

Chương 3

Giao diện người dùng

3.1 Tổng quan thiết kế giao diện

Hệ thống sử dụng giao diện dòng lệnh (Console UI), triển khai toàn bộ trong file `UI/main_console.py`. Lớp `ConsoleUI` **chỉ** giao tiếp với `Scheduler` – tuyệt đối không truy cập trực tiếp vào heap, linked list hay bất kỳ cấu trúc dữ liệu tầng dưới nào.

Luồng hoạt động tổng thể:

1. **Khởi động:** `run()` tạo `ConsoleUI`, gọi `start()`.
2. **Load:** `Scheduler.loadData()` đọc `queue.csv` và `history.csv`, khôi phục toàn bộ trạng thái.
3. **Vòng lặp chính:** Menu hiển thị số liệu trực tiếp, người dùng chọn số chức năng.
4. **Xử lý:** `HANDLERS[choice]()` gọi handler → handler gọi `Scheduler`.
5. **Lưu tự động:** `Scheduler` tự gọi `saveData()` sau mỗi thao tác thay đổi dữ liệu.
6. **Thoát:** Nhập `0` – vòng lặp kết thúc. `KeyboardInterrupt` (Ctrl+C) được bắt bởi `run()` để thoát sạch.

Bảng 3.1: Tóm tắt các chức năng giao diện và phương thức Scheduler tương ứng

Menu	Handler	Scheduler API	Ghi chú
[1] Thêm BN	<code>_menu_add</code>	<code>add_patient()</code>	Tự động sinh mã, validate giờ
[2] Gọi khám	<code>_menu_call_next</code>	<code>call_next()</code>	Peek trước, xác nhận rồi dequeue
[3] Xem hàng đợi	<code>_menu_view_queue</code>	<code>display_queue()</code>	Xuất sorted list từ heap
[4] Tìm kiếm	<code>_menu_search</code>	<code>search_by_id()</code> , <code>search_by_name()</code>	Ba chế độ, $O(1)$ / $O(N)$
[5] Sửa thông tin	<code>_menu_edit</code>	<code>edit_patient()</code>	Heap tự cân bằng lại khi đổi severity
[6] Xóa BN	<code>_menu_delete</code>	<code>remove_patient()</code>	Xác nhận trước khi xóa
[7] Lịch sử	<code>_menu_history</code>	<code>display_history()</code>	Duyệt LinkedList
[8] Thống kê	<code>_menu_stats</code>	<code>get_statistics()</code>	Biểu đồ cột ASCII theo mức độ

3.2 Màn hình menu chính

Khi chạy chương trình, hệ thống tự động gọi `Scheduler.loadData()` để khôi phục dữ liệu từ CSV. Mỗi lần hiển thị menu, hệ thống truy vấn số bệnh nhân đang chờ và đã khám để in ngay dưới tiêu đề:

Listing 3.1: Giao diện menu chính với số liệu trạng thái

```

1  =====
2  BENH VIEN -- QUAN LY HANG DOI
3  Dang cho: 5   |   Da kham: 12
4  =====
5  > [1]  Them benh nhan
6  > [2]  Goi kham (lay uu tien cao nhat)
7  > [3]  Xem hang doi hien tai
8  > [4]  Tim kiem benh nhan
9  > [5]  Sua thong tin benh nhan
10 > [6]  Xoa benh nhan khoi hang doi
11 > [7]  Xem lich su da kham
12 > [8]  Thong ke he thong
13 X [0]  Thoat
14 -----
15 Chon chuc nang: _

```

3.3 Mô tả các chức năng

[1] **Thêm bệnh nhân mới.** Hướng dẫn nhập từng trường theo thứ tự. Trường giờ đến là tùy chọn – nếu bỏ trống, Scheduler tự lấy giờ hệ thống hiện tại. Sau khi thêm thành công, hệ thống in lại thông tin xác nhận kèm mã BN tự động.

[2] **Gọi bệnh nhân vào khám.** Gọi `peek()` để hiển thị trước thông tin bệnh nhân ưu tiên nhất rồi yêu cầu xác nhận – tránh gọi khám nhầm. Chỉ sau khi nhập `y` hệ thống mới thực sự dequeue.

[3] **Xem hàng đợi hiện tại.** Gọi `MinHeap.to_sorted_list()` để xuất danh sách theo đúng thứ tự ưu tiên từ cao xuống thấp ($O(N \log N)$).

[4] **Tìm kiếm bệnh nhân.** Cung cấp ba chế độ: tìm theo mã BN dùng `HashMap` $O(1)$; tìm theo tên hỗ trợ khớp một phần (case-insensitive) duyệt heap $O(N)$; tra cứu lịch sử theo mã BN duyệt `LinkedList` $O(N)$.

[5] **Sửa thông tin bệnh nhân.** Hiển thị giá trị hiện tại trong ngoặc vuông – nhấn Enter để giữ nguyên. Khi sửa `severity`, Scheduler tự động gọi `_heapify_up` / `_heapify_down` để điều chỉnh vị trí.

[6] **Xóa bệnh nhân.** Hiển thị đầy đủ thông tin và yêu cầu xác nhận `y` trước khi xóa. Xóa đồng thời khỏi cả `MinHeap` lẫn `HashMap`.

[7] **Xem lịch sử đã khám.** Duyệt từ `LinkedList` theo thứ tự thời gian thực tế bệnh nhân được gọi, bao gồm giờ khám thực tế.

[8] **Thống kê hệ thống.** Tổng hợp số liệu từ `get_statistics()`, vẽ biểu đồ cột ASCII theo từng mức độ bệnh. Cuối màn hình hiển thị thông tin bệnh nhân tiếp theo bằng `peek()` – không làm thay đổi hàng đợi.

3.4 Xử lý lỗi và kiểm tra đầu vào

Hệ thống áp dụng mô hình xử lý lỗi hai tầng: tầng UI validate dữ liệu thô (kiểu, khoảng giá trị), tầng Scheduler validate nghiệp vụ (định dạng giờ, mã BN trùng). Scheduler trả về tuple (ok: bool, msg: str); UI chỉ cần kiểm tra cờ ok để in thông báo phù hợp.

Bảng 3.2: Các hàm validate đầu vào tại tầng UI

Hàm	Hành vi	Trường hợp lỗi
<code>_input_str(prompt)</code>	Nhập chuỗi, từ chối rỗng	Trả về None, caller in [x] và return
<code>_input_int(prompt, lo, hi)</code>	Nhập số nguyên [lo, hi]	Bắt ValueError và kiểm tra biên, trả về None
<code>_input_optional_str(prompt)</code>	Chuỗi tùy chọn	Trả về None khi Enter, caller giữ nguyên giá trị cũ

Chương 4

Thử nghiệm và đánh giá hiệu năng

4.1 Kịch bản thực nghiệm

So sánh thời gian thực thi của **Binary Heap** với hai cấu trúc đối chứng **Unsorted List** và **Sorted List**.

- **Phương pháp:** Dùng `time.perf_counter()` đo tổng thời gian hoàn thành N thao tác *enqueue* và N thao tác *dequeue*.
- **Dữ liệu:** N bệnh nhân được sinh ngẫu nhiên với đầy đủ thuộc tính để đảm bảo tính khách quan.
- **Số lượng mẫu:** $N \in \{500, 1000, 2000, 5000, 10000\}$.

4.2 Kết quả thực nghiệm

Bảng 4.1: Thời gian thực thi (N enqueue + N dequeue, đơn vị: ms)

N	Binary Heap (ms)	Unsorted List (ms)	Sorted List (ms)
500	2,14	29,34	11,06
1.000	5,04	79,73	30,83
2.000	13,58	237,12	100,60
5.000	30,59	1.647,20	679,18
10.000	81,99	16.122,81	10.128,35

Đo thực tế trên máy tính cá nhân, Python 3.12.

4.3 Phân tích và nhận xét

1. **Binary Heap:** Thể hiện tốc độ vượt trội và cực kỳ ổn định. Khi N tăng gấp 10 lần, thời gian xử lý tăng rất ít, bám sát quy luật tăng trưởng $O(N \log N)$.

2. **Unsorted List** và **Sorted List**: Cả hai đều chậm rõ rệt khi $N \geq 5000$. Thời gian tăng trưởng theo hàm bậc hai $O(N^2)$ (khi N gấp đôi, thời gian tăng khoảng 4 lần).
3. **Kết luận**: Với $N = 10.000$, Binary Heap nhanh hơn Unsorted List hơn **196 lần**. Thực nghiệm xác nhận Binary Heap là lựa chọn đúng đắn cho hệ thống y tế cần phản hồi tức thời.

Chương 5

Kiểm thử chương trình

5.1 Kịch bản kiểm thử

Bảng 5.1: Bảng kịch bản kiểm thử hệ thống (26 test case)

Mã	Mô tả	Đầu vào	Kết quả mong đợi	KQ
TC-01	Insert số nguyên, extract theo thứ tự tăng dần	insert([7,2,9,1,5,3]), extract_min() ×6	Trả về [1,2,3,5,7,9] đúng thứ tự	Đạt
TC-02	Heap có đúng 1 phần tử	insert(42), extract_min()	Trả về 42, heap rỗng sau đó	Đạt
TC-03	peek_min() không thay đổi heap	insert([5,1,3]), peek_min() ×3	Mỗi lần trả về 1, size vẫn bằng 3	Đạt
TC-04	Xóa nút lá	insert([1,3,5,7,9]), delete(x==9)	removed=9, còn lại [1,3,5,7]	Đạt
TC-05	Xóa nút gốc	insert([1,3,5,7,9]), delete(x==1)	removed=1, còn lại [3,5,7,9]	Đạt
TC-06	Xóa phần tử không tồn tại	insert([2,4,6]), delete(x==99)	Trả về None, size không đổi	Đạt
TC-07	find_all() tìm phần tử thỏa điều kiện	insert([1..8]), find_all(x%2==0)	Trả về [2,4,6,8]	Đạt
TC-08	to_sorted_list() không phá vỡ heap	insert([4,2,6,1]), to_sorted_list()	lst=[1,2,4,6], heap giữ nguyên	Đạt
TC-09	Severity nhỏ hơn được ưu tiên hơn	BN001(sev=1), BN002(sev=3)	BN001 ưu tiên hơn BN002	Đạt

(tiếp theo)

Mã	Mô tả	Đầu vào	Kết quả mong đợi	KQ
TC-10	Cùng severity, đến sớm hơn được ưu tiên	BN001(arr=07:30), BN002(arr=09:00)	BN001 ưu tiên hơn BN002	Đạt
TC-11	Cùng sev + arr, reg_order nhỏ hơn được ưu tiên	reg=5 và reg=10, cùng sev và arr	BN reg=5 ưu tiên hơn	Đạt
TC-12	__eq__ chỉ so sánh patient_id	Hai bệnh nhân cùng ID	p1 == p2 trả về True	Đạt
TC-13	Validate giờ đúng chuẩn HH:MM	13:45	(True, "13:45")	Đạt
TC-14	Chuẩn hóa H:MM thành 0H:MM	7:05	(True, "07:05")	Đạt
TC-15	Từ chối giờ có phút > 59	10:61	False, báo lỗi	Đạt
TC-16	Từ chối chuỗi không phải giờ	"buoi sang"	False, báo lỗi	Đạt
TC-17	Tự gán reg_order khi enqueue	reg_order=0	Tự động gán ≥ 1	Đạt
TC-18	Giữ nguyên reg_order đã có	reg_order=77	Không bị ghi đè	Đạt
TC-19	size() và is_empty() nhất quán	Queue mới	size=0, is_empty=True	Đạt
TC-20	Dequeue đúng thứ tự ưu tiên	3 BN khác severity	$B \rightarrow C \rightarrow A$	Đạt
TC-21	contains() trước và sau dequeue	enqueue, dequeue	True trước, False sau	Đạt
TC-22	remove_by_id với ID không tồn tại	BN_KHONG_TON_TAI	Trả về None	Đạt
TC-23	clear() xóa toàn bộ hàng đợi	enqueue 4 BN	size=0, is_empty=True	Đạt
TC-24	Reinsert sau sửa severity	A: severity 5 $\rightarrow$ 1	A nhảy lên đầu hàng đợi	Đạt
TC-25	get_sorted_list() không làm đổi heap	enqueue 3 BN	Top và size giữ nguyên	Đạt

(tiếp theo)

Mã	Mô tả	Đầu vào	Kết quả mong đợi	KQ
TC-26	save/load giữ toàn vẹn dữ liệu	Lưu rồi đọc lại file	Thứ tự phục hồi 100%	Đạt

5.2 Kết quả kiểm thử

Toàn bộ 26 kịch bản kiểm thử thuộc ba nhóm **MinHeap**, **Patient** và **PriorityQueue** đều đạt kết quả mong đợi. Các phép kiểm thử xác nhận tính đúng đắn của các thao tác trên MinHeap, quy tắc ưu tiên ba cấp (*severity* $\rightarrow$ *arrival_time* $\rightarrow$ *reg_order*), cơ chế quản lý hàng đợi ưu tiên, cũng như tính toàn vẹn dữ liệu khi lưu và khôi phục từ file CSV.

Chương 6

Kết luận và hướng phát triển

6.1 Kết luận

Nhóm đã xây dựng thành công hệ thống quản lý hàng đợi bệnh viện với kiến trúc phân tầng rõ ràng. Điểm cốt lõi là **Binary Min-Heap** tự cài đặt cho hàng đợi ưu tiên đạt hiệu suất ổn định $O(\log N)$, kết hợp **HashMap** tra cứu $O(1)$ và **LinkedList** lưu lịch sử $O(1)$. Trạng thái dữ liệu được bảo toàn qua cơ chế đồng bộ file tự động `loadData()` và `saveData()`. Kết quả thực nghiệm hoàn toàn khớp với phân tích lý thuyết và chứng minh độ ưu việt của mô hình thuật toán lựa chọn.

6.2 Hạn chế và hướng phát triển

Hạn chế hiện tại:

- Giao diện console còn đơn giản, chưa thân thiện.
- HashMap bọc dict Python – chưa tự cài đặt hàm băm thủ công hoàn toàn.
- Chưa xử lý đồng thời (concurrent access) khi nhiều người dùng cùng lúc.

Hướng phát triển:

- Xây dựng giao diện đồ họa (Tkinter hoặc web-based).
- Tự cài đặt HashMap với separate chaining hoàn toàn thủ công.
- Tích hợp cơ sở dữ liệu SQL thay thế CSV.
- Mở rộng thống kê: thời gian chờ trung bình, biểu đồ tần suất theo giờ.

Chương 7

Phân công và đánh giá nhóm

Bảng 7.1: Bảng phân công chi tiết công việc thành viên

Hạng mục	Phụ trách	Hoàn thành
Chương 1 – Phân tích bài toán	Mai Đức	100%
Chương 2 – Kiến trúc tổng thể & Class Diagram	Huy Đức	100%
Chương 2 – Lớp Patient & MinHeap	Huy Đức	100%
Chương 2 – LinkedList & HashMap	Tuấn Đạt	100%
Chương 2 – Giải mã thuật toán (enqueue / dequeue)	Bá Dũng	100%
Chương 2 – Cài đặt chương trình (code)	Huy Đức	100%
Chương 2 – Phân tích độ phức tạp	Hải Dương	100%
Chương 3 – Giao diện người dùng	Tuấn Đạt	100%
Chương 4 – Thử nghiệm & đánh giá hiệu năng	Hải Dương	100%
Chương 5 – Kiểm thử (kịch bản test case)	Mai Đức	100%
Chương 6 – Tổng hợp kết luận	Bá Dũng	100%
Tổng hợp, đóng gói & nộp bài	Cả nhóm	100%
Video thuyết trình (≤ 20 phút)	Cả nhóm	100%

Tài liệu tham khảo

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
2. Đỗ Xuân Lôi. *Cấu trúc dữ liệu và giải thuật*. NXB Khoa học và Kỹ thuật.
3. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). *Data Structures and Algorithms in Python*. Wiley.
4. Python Software Foundation. *heapq – Heap queue algorithm*. <https://docs.python.org/3/library/heapq.html>